Betriebssysteme

2. Prozesse und Threads
Grundlegende Modellierung

Prof. Dr. Harald, Kornmayer
Institut für Informatik, DHBW Mannheim, Germany

Prozesse und Threads

- Agenda
  - Prozesse
  - Threads
  - Interprozess-Kommunikation
  - Klassische Probleme bei Interprozess-Kommunikation
  - Scheduling
  - Deadlocks /Verklemmungen

Aufgabe

- Definieren Sie den Begriff Prozess!
  - Was ist der Unterschied zwischen Programm und Prozess?
    - Suchen Sie ein Analogon!
  - Was realisiert das Betriebssystem durch Prozesse?
  - Welche Daten brauchen Sie, um einen Prozess zu beschreiben?
  - 3 er Team
  - 5 Minuten



Prozesse

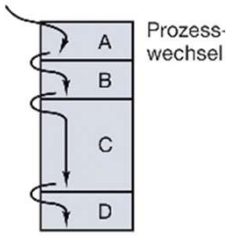
- Das **zentrale Konzept** in jedem Betriebssystem
  - **Prozess** = ein Programm in Ausführung
  - Prozesse modellieren Nebenläufigkeit
    - unterstützen (Quasi-)Parallelität
    - Multiprogrammierung
      - Schnelles Hin und Herschalten zwischen Prozessen
      - Umladen der Register notwendig!
- **Prozessmodell**
  - „*Das Betriebssystem ist als Menge von (sequenziellen) Prozessen organisiert*“
  - jeder Prozess umfasst eine virtuelle CPU mit eigenem Adressraum
    - Befehlszähler, Registerinhalte, Belegung von Variablen
    - Und einiges mehr!

Prozesse

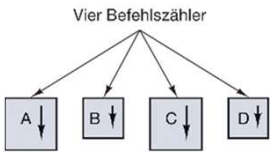
- Unterschied Programm und Prozess
  - Vergleich:
    - Programm ist das Kuchenrezept mit allen Anweisungen
    - Prozess ist die Aktivität des Backen
    - Beachte: Ein Programm 2x starten, ergibt 2 Prozesse!

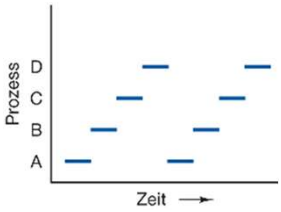
Prozesse

Es gibt nur einen Befehlszähler im System



Es gibt mehrere logische Befehlszähler im System
- einen pro Prozess





- Ein Prozess muss zu Ende laufen
- Nebenläufigkeit nicht möglich

- Prozesse können nebenläufig ausgeführt werden
- Quasi-Gleichzeitig

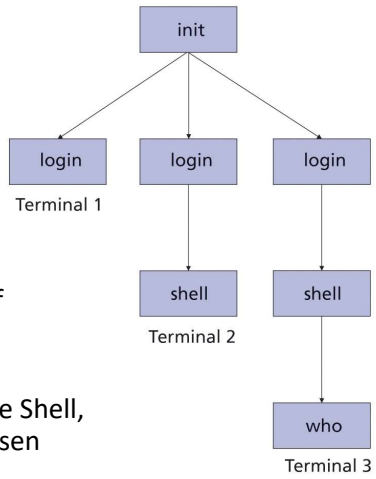
Konsequenz:

Laufzeit eines Programms ist nicht mehr reproduzierbar!

→ Programme dürfen keine Annahmen über den Zeitablauf enthalten

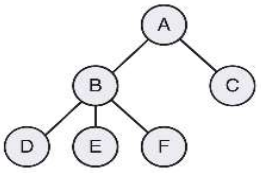
Prozesse

- **Wie wird ein Prozess erzeugt?**
  - durch das Betriebssystem
    - Initialisierung
      - Als Hintergrundprozess
    - durch einen anderen Prozess
      - Systemaufruf
  - Bsp: Linux starten
    - init liest Datei /etc/init/tty?.conf
    - Startet dann ein login für jedes Terminal
    - Nach der Anmeldung startet eine Shell, von der aus die nächsten Prozessen gestartet werden können.



Prozesse

- Beziehungen zwischen Prozessen
  - Eltern-Kind Beziehung
    - Entstehung von Prozess-Familien bzw. Prozess-Hierarchien
  - UNIX
    - Signale werden an alle Mitglieder der Familie verschickt
      - Prozess entscheiden wie auf Signal reagiert werden!
  - Windows
    - Kein Konzept der Prozesshierarchie
    - Alle Prozesse sind gleichwertig
      - Beziehungen werden durch spezielle Tokens (Handle) gesteuert!
        - » Weitergabe von Handles möglich (Enterbung)

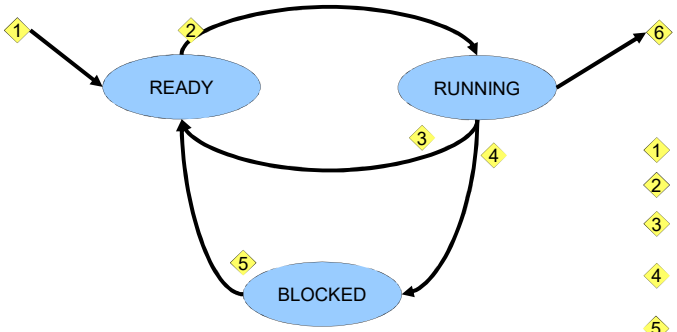


Prozesse und Threads

- **Prozessbeendigung**
  - **Wie werden Prozesse beendigt?**
  - Normales Beenden (freiwillig)
    - Wenn Aufgabe erledigt ist! (exit bzw. exitProcess)
  - Beenden aufgrund eines Fehlers (freiwillig)
    - Der Prozess stellt einen Fehler fest, der nicht vom Programm verursacht wurde (z.b. Datendatei nicht vorhanden)
  - Beenden aufgrund eines schwerwiegenden Fehlers (unfreiwillig)
    - Der Prozess selbst verursacht einen Fehler
    - Ausführen eines unzulässigen Befehl
    - Zugriff auf ungültige Speicheradresse
  - Beenden durch anderen Prozess (unfreiwillig)
    - (kill bzw. TerminateProcess)

Prozesse - Zustandsmodell

- Prozess-Zustände
  - wichtig für die Verwaltung der Ressourcen durch BS



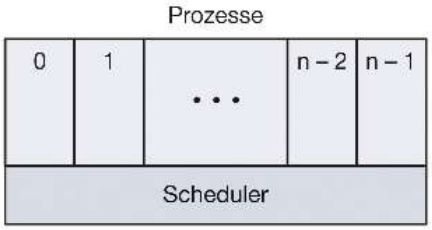
1 Start nach Erzeugung
2 Auswahl durch Scheduler
3 Prozess gibt CPU frei
4 Prozess blockiert, weil er auf Eingabe wartet
5 Eingabe vorhanden (durch Interupt signalisiert)
6 Beendigung des Prozesses

Aktivierung des Schedulers bei Ereignissen 1 3 4 5 6

Ergebnis des Schedulers 2

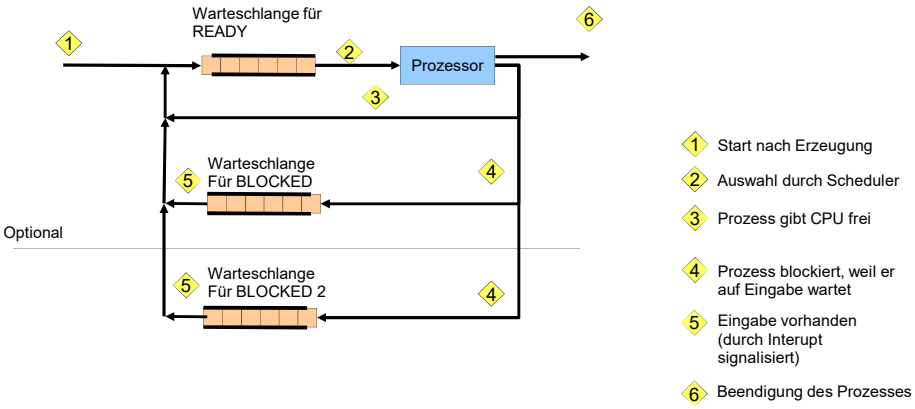
Prozesse

- Scheduler
  - unterste Schicht des Betriebssystems
  - reagiert auf
    - Systemaufrufe
      - z.B. beim Starten (fork) und Stoppen (kill) von Prozessen
    - Unterbrechungen (Interrup
    - …
  - verwaltet die Prozesse in den Warteschlangen
    - **Wesentlich: entscheidet wer als nächstes rechnen darf!**

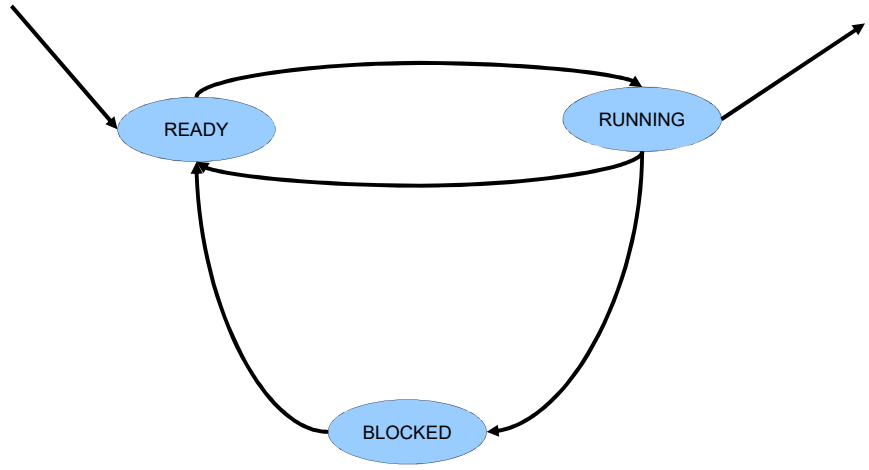


Prozesse

- Scheduler
  - Warteschlangen



Prozesse - Simulation

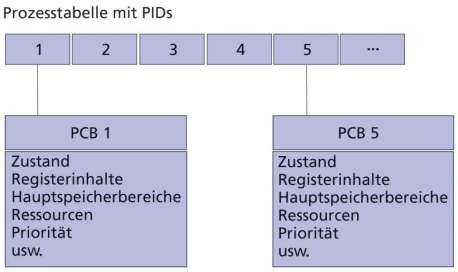


Prozesse

- Datenmodell zur Verwaltung von Prozessen
  - Prozesstabelle
    - Ein Eintrag pro Prozess (Prozesskontrolblock (PCB))
    - PCB enthält alle Information über Prozess
      - Alle Informationen → Datenmodell
      - Zur **Prozessverwaltung**
        - » Prozessidentifikation
          - Kennung des Prozesses (P-ID) und des Elternprozesses
          - Benutzerkennung
          - Prozessinformation
            - Priorität, verbrauchte CPU-Zeit, PSW, Signale,…
        - » Ausführungs-/Zustandsinformation
          - Zustand, Register, Befehlszähler, Stack, …
      - Zur **Speicherverwaltung**
        - » Zeiger auf Textsegment, Datensegment, Stacksegment
      - Zur **Dateiverwaltung**
        - » Wurzelverzeichnis, Arbeitsverzeichnis, offene Dateien, Benutzer-Id, Gruppen-ID, …

Prozesse

- Prozesswechsel
  - mit Hilfe der Prozesstabelle
    - Bsp: Interrupt von E/A-Gerät („Daten stehen bereit“)



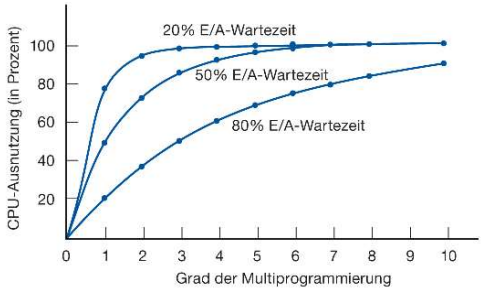
    - Wechseln zwischen zwei PCB

Prozesse

- Prozesswechsel
  - Ablauf
    - Sichern des Befehlszählers, Prozessorstatus, etc
      - Prozess auf „Blocked“ setzen
    - Ursache der Unterbrechung ermitteln
    - Ereignis (z.B. Ende der E/A) entsprechend behandeln
      - Blockierte Threads auf „READY“ setzen
    - Sprung zum Scheduler
    - Entscheidet welcher Prozess als nächstes läuft

Prozesse

- Modell der Multiprogrammierung
  - Vorteil: Verbesserung der CPU Auslastung
    - N: Prozesse die gleichzeitig im Speicher gehalten werden
    - P: E/A- Anteil eines Prozesses
      - z.B. 20% der Zeit warten auf Daten/Festplatte



CPU-Ausnutzung = $1- P^N$

Mehr Speicher ermöglicht mehr Prozesse

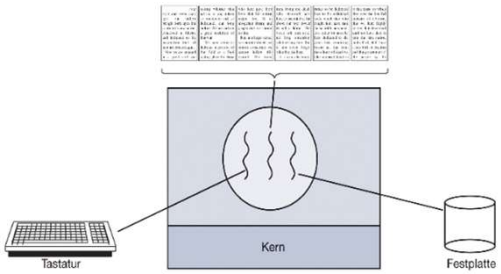
→ Verbesserung der Auslastung

Aufgabe

- Im bisherigen Prozessmodel hält jeder Prozess Information über seinen Adressraum und über Dateien
- Ein Textverarbeitungsprogramm soll in der Lage sein Eingaben über die Tastatur entgegenzunehmen, den formatierten Text auf dem Bildschirm auszugeben und gleichzeitig ein Sicherung auf die Festplatte zu schreiben.
- Was sind die Vor- und Nachteile des bisherigen Prozessmodells für dieses Scenario?
  - 3er Team
  - 5 Minuten



Prozesse und Threads

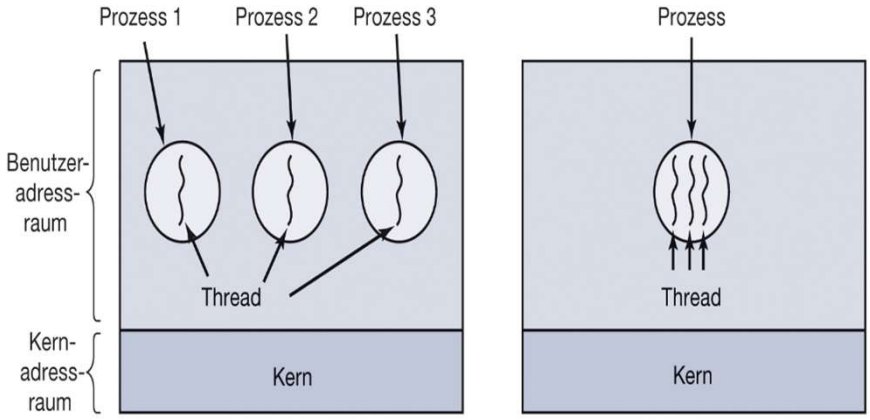
- Prozessmodell
  - Jeder Prozess hat eigenen Adressraum
  - mit einen Ausführungsfaden (thread of control)
  - Fragen:
    - Wieso nicht quasi-parallel den Adressraum bearbeiten?
    - Wieso nicht mehrere Ausführungsfäden im selben Adressraum ablaufen lassen?
- 

Prozesse und Threads

- Threads
  - Erweitertes Prozessmodel
    - Mehrere Ausführungsfäden mit gleichem Adressraum
      - Daten gemeinsam benutzen
    - Ausführung im Benutzermodus möglich
    - Erstellung ist 10-100 mal schneller als Prozesse
  - Konzept der Ressourcen-Bündelung für Prozesse
    - Adressraum, geöffnete Dateien, Signale, Kindprozesse, …
  - Konzept der Ausführung von Prozessen (Threads)
    - Befehlszähler, Register für lokale Variablen, Stack,
    - Mehrere Ausführungsfäden sind nun möglich (Multi-Threading)
      - Hardware-Unterstützung (schnelles Umschalten in nsec)

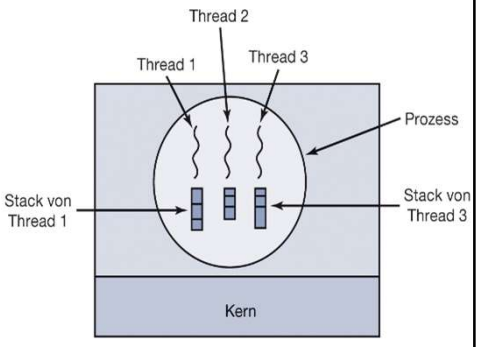
Prozesse und Threads

- Threads



Prozesse und Threads

- Threads
  - Thread-Zustände
    - Gleiche Zustände wie Prozesse
      - RUNNING, BLOCKED, READY
  - Thread-Verwaltung
    - Thread-Tabelle
    - Befehlszähler, Register, Stack und Zustand
      - Jeder Thread hat seinen eigenen Stack

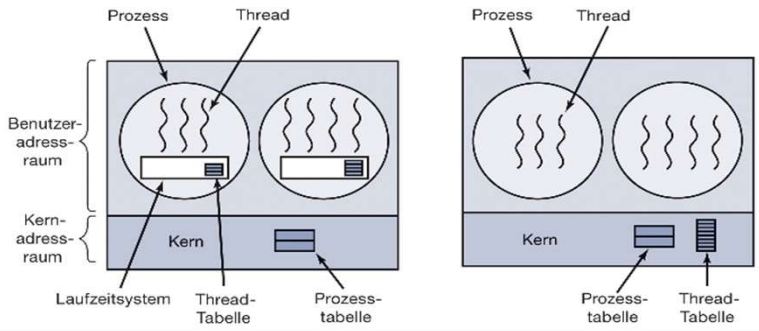


Prozesse und Threads

- Threads
  - Lebenszyklus
    - Erzeugung von Threads
      - Prozess startet einen Thread, der neue Threads erzeugt (create)
        - » Argument: Name der Prozedur, die ausgeführt werden soll
        - » Keine Angaben des Adressraums notwendig
          - Neuer Thread läuft im Adressraum des erzeugenden Threads
    - Beendigung
      - Nach der Beendigung der Aufgabe durch den Thread selbst (exit)
    - Synchronisation
      - Warten auf die Beendigung eines bestimmten Threads (join)
    - Freiwilliger Verzicht
      - Ressourcen an die CPU zurückgeben (yield)
        - » Scheduler beauftragen anderen Thread zustarten

Prozesse und Threads

- Threads
  - Wo läuft der Thread-Scheduler/die Thread-Verwaltung?
    - Zwei Varianten
      - Im Benutzeradressraum/-modus
      - Im Kernadressraum/-modus



Prozesse und Threads

- Threads
  - Wo läuft der Thread-Scheduler/die Thread-Verwaltung?
- Benutzermodus
  - Pro
    - Schneller Wechsel
    - Angepasster Scheduler möglich
  - Con
    - Umgang mit blockierenden Systemaufrufen
    - Ein Thread kann den gesamten Prozess blockieren
      - z.B. bei Seitenfehlern
    - Threads sollen Programme ermöglichen, die oft blockieren
- Kernmodus
  - Pro
    - Kein Laufzeitsystem
    - Effizienter Umgang mit blockierende Systemaufrufen
  - Con
    - Höher Kosten bei Thread- Wechsel durch Systemaufrufe
    - Umgang mit Signalen

- Threads werden meist im Kernmodus realisiert!
- Spannendes Thema in Betriebssystementwicklung

Prozesse und Threads

- Threads
  - Threadwechsel
    - erfolgt immer dann, wenn BS die Kontrolle erhält
      - Bei Systemaufrufen
        - » Thread gibt Kontrolle freiwillig ab!
      - Bei Ausnahme
      - Bei Interrupt
        - » Periodischer Timer-Interrupt stellt sicher, daß kein Thread die CPU monopolisiert
    - Scheduler des Betriebssysteme entscheidet, welcher Thread als nächstes rechnen soll
      - Falls nötig, wird dann auch der Prozess gewechselt
      - Scheduler kann diese Kosten berücksichtigen

Prozesse und Threads

- Zusammenfassung
  - Prozessmodell
    - *„Das Betriebssystem ist als Menge von (sequenziellen) Prozessen organisiert“*
      - eine Liste und für jeden Prozess einen Eintrag
  - **Datenmodell** von Prozessen und Threads
    - Prozess: Einheit der Ressourcenverwaltung, Schutzeinheit
      - PID, UserID, Adressraum, geöffnete Dateien, Signale, Prioritäten, ...
    - Thread: Einheit der Prozessorzuteilung
      - Befehlszähler, CPU-Register, Stack, Thread-Zustand, …

      → mehrere Threads pro Prozess möglich
  - **Zustandsmodell** für Prozesse und Threads
    - RUNNING, READY, BLOCKED
    - Realisierung durch Warteschlangen

Prozesse und Threads

- Zusammenfassung
  - Realisierungsvarianten für Threads
    - Im Kernmodus (gängig) , im Benutzermodus
    - Threadwechsel
      - Bei Systemaufrufen, Ausnahmen oder Interrupts
      - Umladen des Prozessorkontext
        - Beim Prozesswechsel auch Wechsel des Speicherabbilds
  - Programmier-Schnittstellen
    - POSIX-Threads
    - JAVA Threads